

# News App - Complete Project Documentation

A comprehensive guide for Android beginners to understand, navigate, and extend this News application.

## Table of Contents

- 1. [Project Overview](#)
- 2. [Getting Started](#)
- 3. [Folder and File Structure](#)
- 4. [App Architecture](#)
- 5. [User Flows](#)
- 6. [Screen-by-Screen Guide](#)
- 7. [Code-Level Function Reference](#)
- 8. [Database Implementation](#)
- 9. [Network Layer](#)
- 10. [Dependency Injection](#)
- 11. [Data Flow Diagrams](#)
- 12. [How to Add New Features](#)
- 13. [Common Patterns Used](#)
- 14. [Troubleshooting](#)

## 1. Project Overview

### What is this app?

This is a **News Reader Application** built with modern Android development practices. It allows users to:

- View top news headlines from around the world
- Read detailed articles
- Save favorite articles for later reading
- Browse saved favorites

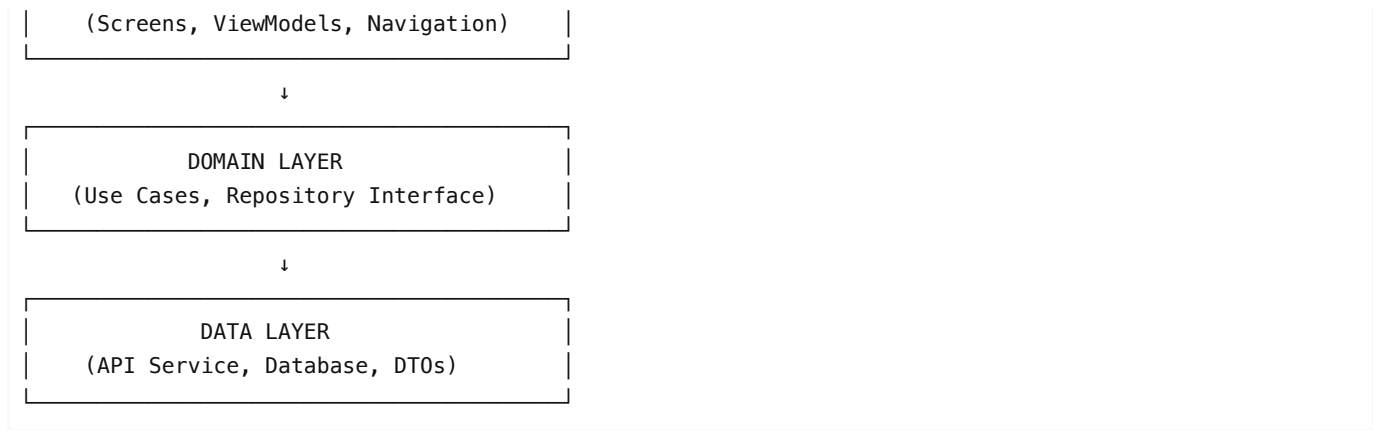
### Technology Stack

| Technology      | Purpose                               |
|-----------------|---------------------------------------|
| Kotlin          | Programming language                  |
| Jetpack Compose | Modern UI framework (no XML layouts!) |
| Room            | Local SQLite database                 |
| Retrofit        | Network HTTP client                   |
| Hilt            | Dependency injection                  |
| Coroutines      | Asynchronous programming              |
| Flow            | Reactive data streams                 |
| Material3       | Design system                         |

### Architecture Pattern

This app uses **Clean Architecture** with **MVVM (Model-View-ViewModel)**:





## 2. Getting Started

### Prerequisites

- Android Studio (Arctic Fox or newer)
- JDK 17 or higher
- An Android device/emulator (API 24+)

### Running the App

1. Open the project in Android Studio
2. Wait for Gradle sync to complete
3. Click "Run" (green play button)
4. Use demo credentials to log in:
  - Email: `test@example.com`
  - Password: `password`

### API Key

The app uses [NewsAPI.org](https://newsapi.org/) for fetching news. The API key is currently hardcoded in `NetworkModule.kt`. For production, you should:

1. Get your own API key from [newsapi.org](https://newsapi.org/)
2. Store it in `local.properties` (never commit this file)
3. Access it via `BuildConfig`

## 3. Folder and File Structure

### Complete Project Tree

```
News/
├── app/
│   ├── src/
│   │   ├── main/
│   │   │   ├── java/com/example/newsapp/
│   │   │   │   ├── data/ # DATA LAYER
│   │   │   │   │   ├── api/
│   │   │   │   │   │   └── NewsApiService.kt
│   │   │   │   │   ├── db/
│   │   │   │   │   │   ├── AppDatabase.kt
│   │   │   │   │   │   ├── dao/
│   │   │   │   │   │   │   └── ArticleDao.kt
│   │   │   │   │   └── entity/
```

```
└─ ArticleEntity.kt
└─ model/
  └─ NewsDtos.kt
└─ repository/
  └─ NewsRepositoryImpl.kt
└─ Mappers.kt

└─ domain/                                # DOMAIN LAYER
  └─ model/
    └─ Article.kt
  └─ repository/
    └─ NewsRepository.kt
  └─ usecase/
    └─ NewsUseCases.kt

└─ ui/                                    # PRESENTATION LAYER
  └─ navigation/
    └─ NavGraph.kt
    └─ Screen.kt
  └─ login/
    └─ LoginScreen.kt
  └─ home/
    └─ HomeScreen.kt
    └─ components/
      └─ ArticleItem.kt
      └─ Shimmer.kt
  └─ detail/
    └─ DetailScreen.kt
  └─ favorites/
    └─ FavoritesScreen.kt
  └─ theme/
    └─ Color.kt
    └─ Theme.kt
    └─ Type.kt

└─ viewmodel/                            # VIEWMODELS
  └─ HomeViewModel.kt
  └─ DetailViewModel.kt
  └─ FavoritesViewModel.kt
  └─ LoginViewModel.kt

└─ di/                                    # DEPENDENCY INJECTION
  └─ AppModule.kt
  └─ DatabaseModule.kt
  └─ NetworkModule.kt

└─ MainActivity.kt                        # ENTRY POINT
└─ NewsApp.kt                            # APPLICATION CLASS

└─ res/                                   # RESOURCES
  └─ drawable/
  └─ mipmap-*/
  └─ values/
    └─ colors.xml
    └─ strings.xml
    └─ themes.xml
```



Folder Explanations

data/ - Data Layer

Purpose: Handles all data operations (API calls, database access)

| Folder/File                      | Role                                        |
|----------------------------------|---------------------------------------------|
| api/NewsApiService.kt            | Defines HTTP endpoints for News API         |
| db/AppDatabase.kt                | Room database configuration                 |
| db/dao/ArticleDao.kt             | Database operations (insert, delete, query) |
| db/entity/ArticleEntity.kt       | Database table structure                    |
| model/NewsDtos.kt                | Data Transfer Objects for API responses     |
| repository/NewsRepositoryImpl.kt | Coordinates API and database                |
| Mappers.kt                       | Converts between data models                |

domain/ - Domain Layer

Purpose: Contains business logic and rules

| Folder/File                  | Role                                    |
|------------------------------|-----------------------------------------|
| model/Article.kt             | Core business model used throughout app |
| repository/NewsRepository.kt | Interface defining data operations      |
| usecase/NewsUseCases.kt      | Individual business operations          |

ui/ - Presentation Layer

Purpose: Everything the user sees and interacts with

| Folder/File | Role                  |
|-------------|-----------------------|
| navigation/ | App navigation setup  |
| login/      | Login screen          |
| home/       | Main news feed screen |
| detail/     | Article detail screen |

|            |                                 |
|------------|---------------------------------|
| favorites/ | Saved articles screen           |
| theme/     | Colors, typography, and theming |

**viewModel/** - State Management

**Purpose:** Holds UI state and handles user actions

| File                  | Screen it serves |
|-----------------------|------------------|
| LoginViewModel.kt     | LoginScreen      |
| HomeViewModel.kt      | HomeScreen       |
| DetailViewModel.kt    | DetailScreen     |
| FavoritesViewModel.kt | FavoritesScreen  |

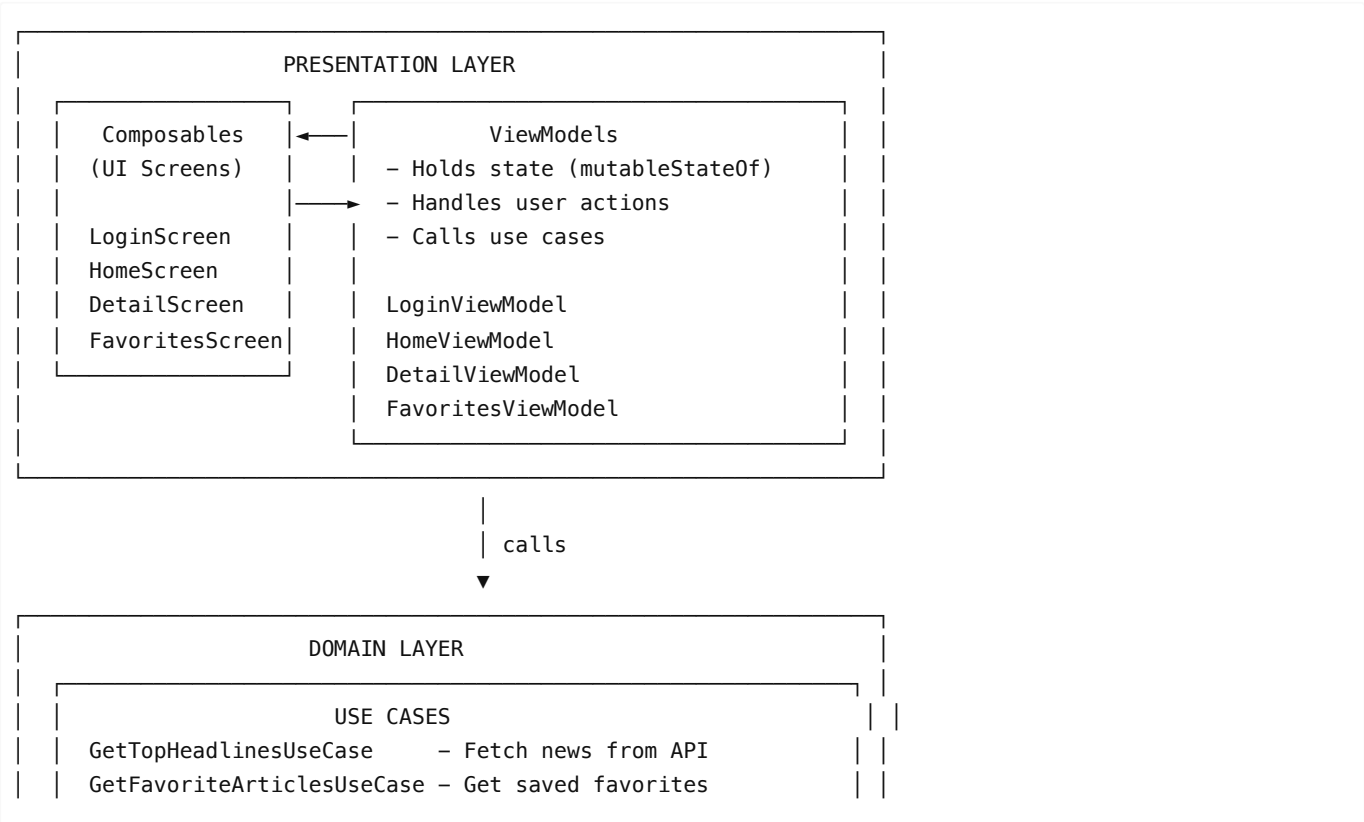
**di/** - Dependency Injection

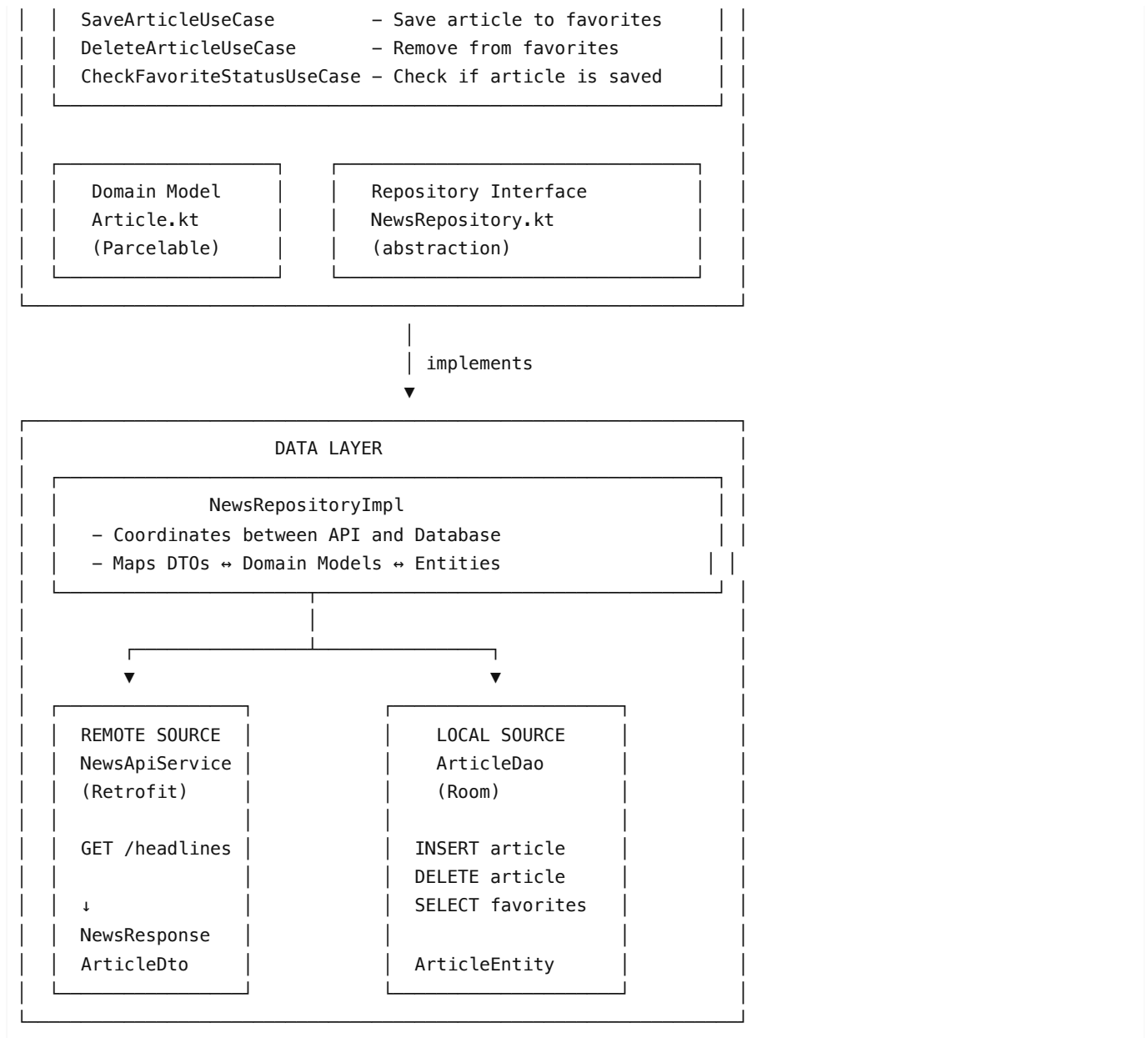
**Purpose:** Provides dependencies to classes that need them

| File              | What it provides                 |
|-------------------|----------------------------------|
| NetworkModule.kt  | Retrofit, OkHttp, NewsApiService |
| DatabaseModule.kt | Room database, ArticleDao        |
| AppModule.kt      | Repository binding               |

# 4. App Architecture

## Clean Architecture Layers



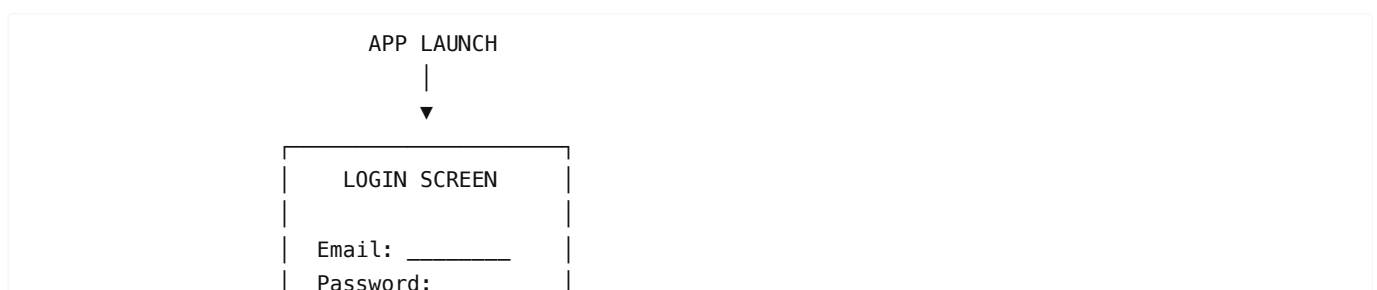


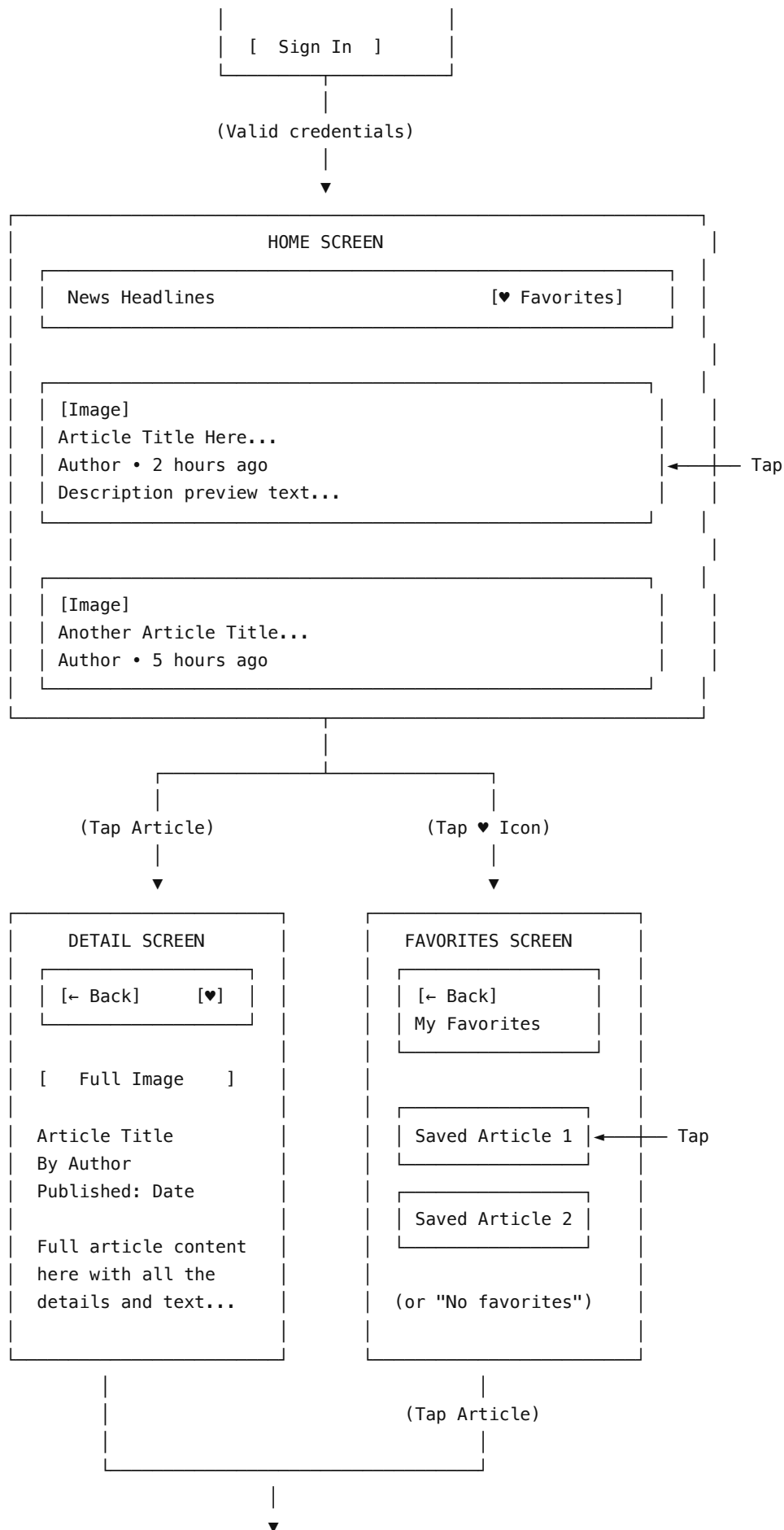
## Why Clean Architecture?

1. **Testability:** Each layer can be tested independently
2. **Maintainability:** Changes in one layer don't affect others
3. **Scalability:** Easy to add new features
4. **Separation of Concerns:** Each class has one responsibility

## 5. User Flows

### Complete App Navigation





DETAIL SCREEN  
(same as above)

### Flow 1: Login Flow

1. User opens app
2. LoginScreen displays
3. User enters email and password
4. User taps "Sign In"
5. LoginViewModel.onLoginClick() validates credentials
6. If valid → Navigate to HomeScreen (LoginScreen removed from back stack)
7. If invalid → Show error dialog

### Flow 2: View News Flow

1. HomeScreen displays
2. HomeViewModel.init() automatically called
3. getNews() fetches articles from API
4. While loading: Shimmer animation shows
5. On success: Articles display in list
6. On error: Error card displays

### Flow 3: Save Favorite Flow

1. User taps article → DetailScreen opens
2. DetailViewModel receives article
3. isFavorite status checked from database
4. Heart icon shows outline (not saved) or filled (saved)
5. User taps heart icon
6. onFavoriteClick() toggles status
7. Optimistic update: UI updates immediately
8. Database updated in background
9. FavoritesScreen automatically updates

## 6. Screen-by-Screen Guide

### LoginScreen ( `ui/login/LoginScreen.kt` )

**Purpose:** Authenticate users before accessing the app

**Visual Elements:**

- Gradient background (dark navy to blue)
- 12 floating animated news-related icons
- Email input field
- Password input field
- "Sign In" button with gradient
- Error dialog for failed login

**ViewModel:** `LoginViewModel`

**Key Functions:**

```
// Update email field
fun onEmailChange(newEmail: String)
```

```
// Update password field
fun onPasswordChange(newPassword: String)

// Validate and attempt login
fun onLoginClick(onSuccess: () -> Unit)

// Clear error message
fun clearError()
```

#### Demo Credentials:

- test@example.com / password
- parag@icici.com / password

---

## HomeScreen ( ui/home/HomeScreen.kt )

**Purpose:** Display list of top news headlines

#### Visual Elements:

- Gradient header with "News Headlines" title
- Favorites icon (heart) in top right
- Scrollable list of article cards
- Shimmer loading animation
- Error state with retry option

**ViewModel:** HomeViewModel

#### State:

```
data class HomeState(
    val isLoading: Boolean = false,
    val articles: List<Article> = emptyList(),
    val error: String? = null
)
```

#### Key Functions:

```
// Called automatically when ViewModel is created
init {
    getNews()
}

// Fetch news from API
private fun getNews() {
    viewModelScope.launch {
        _state.value = HomeState(isLoading = true)

        getTopHeadlinesUseCase(page = 1, pageSize = 20)
            .onSuccess { articles ->
                _state.value = HomeState(articles = articles)
            }
            .onFailure { error ->
                _state.value = HomeState(error = error.message)
            }
    }
}
```

```
}  
}
```

#### Navigation:

- Tap article → DetailScreen
- Tap favorites icon → FavoritesScreen

---

## DetailScreen ( `ui/detail/DetailScreen.kt` )

**Purpose:** Show full article details and allow favoriting

#### Visual Elements:

- Back arrow in top bar
- Heart icon (favorite toggle) in top bar
- Full-width article image
- Article title
- Author and publication date
- Full article content (scrollable)

**ViewModel:** `DetailViewModel`

#### State:

```
data class DetailState(  
    val isFavorite: Boolean = false  
)  
  
// Article stored separately as StateFlow  
private val _article = MutableStateFlow<Article?>(null)  
val article: StateFlow<Article?> = _article.asStateFlow()
```

#### Key Functions:

```
// Initialize with article and observe favorite status  
fun initArticle(article: Article) {  
    _article.value = article  
    observeFavoriteStatus(article.url)  
}  
  
// Observe database for favorite status changes  
private fun observeFavoriteStatus(url: String) {  
    favoriteStatusJob?.cancel()  
    favoriteStatusJob = viewModelScope.launch {  
        checkFavoriteStatusUseCase(url).collect { isFavorite ->  
            _state.value = _state.value.copy(isFavorite = isFavorite)  
        }  
    }  
}  
  
// Toggle favorite status (with optimistic update)  
fun onFavoriteClick() {  
    val currentArticle = _article.value ?: return  
  
    // Optimistic update - UI updates immediately  
    val newFavoriteStatus = !_state.value.isFavorite
```

```

        _state.value = _state.value.copy(isFavorite = newFavoriteStatus)

// Database update in background
viewModelScope.launch {
    if (newFavoriteStatus) {
        saveArticleUseCase(currentArticle.copy(isFavorite = true))
    } else {
        deleteArticleUseCase(currentArticle)
    }
}
}

```

## FavoritesScreen ( ui/favorites/FavoritesScreen.kt )

**Purpose:** Display saved favorite articles

**Visual Elements:**

- Gradient header matching HomeScreen
- Back arrow to return to HomeScreen
- List of favorited articles
- Empty state message when no favorites

**ViewModel:** FavoritesViewModel

**State:**

```

data class FavoritesState(
    val articles: List<Article> = emptyList()
)

```

**Key Functions:**

```

// Called automatically when ViewModel is created
init {
    getFavoriteArticles()
}

// Observe favorites from database (real-time updates)
private fun getFavoriteArticles() {
    viewModelScope.launch {
        getFavoriteArticlesUseCase().collect { favorites ->
            _state.value = FavoritesState(articles = favorites)
        }
    }
}

```

**Real-time Updates:** When you favorite/unfavorite an article in DetailScreen, the FavoritesScreen list updates automatically because it observes the database using Flow.

## 7. Code-Level Function Reference

### Use Cases ( domain/usecase/NewsUseCases.kt )

Use cases encapsulate single business operations. Each use case has one responsibility.

### GetTopHeadlinesUseCase

```
class GetTopHeadlinesUseCase @Inject constructor(
    private val repository: NewsRepository
) {
    suspend operator fun invoke(
        page: Int = 1,
        pageSize: Int = 20
    ): Result<List<Article>> {
        return try {
            val articles = repository.getTopHeadlines(page, pageSize)
            Result.success(articles)
        } catch (e: Exception) {
            Result.failure(e)
        }
    }
}
```

**What it does:** Fetches news headlines from the API **Returns:** `Result<List<Article>>` - Success with articles or Failure with error

---

### GetFavoriteArticlesUseCase

```
class GetFavoriteArticlesUseCase @Inject constructor(
    private val repository: NewsRepository
) {
    operator fun invoke(): Flow<List<Article>> {
        return repository.getFavoriteArticles()
    }
}
```

**What it does:** Returns a Flow that emits favorite articles whenever they change **Returns:** `Flow<List<Article>>` - Reactive stream of favorites

---

### SaveArticleUseCase

```
class SaveArticleUseCase @Inject constructor(
    private val repository: NewsRepository
) {
    suspend operator fun invoke(article: Article) {
        repository.saveArticle(article)
    }
}
```

**What it does:** Saves an article to the local database as a favorite

---

### DeleteArticleUseCase

```
class DeleteArticleUseCase @Inject constructor(
    private val repository: NewsRepository
) {
    suspend operator fun invoke(article: Article) {
```

```

        repository.deleteArticle(article)
    }
}

```

**What it does:** Removes an article from favorites

---

### CheckFavoriteStatusUseCase

```

class CheckFavoriteStatusUseCase @Inject constructor(
    private val repository: NewsRepository
) {
    operator fun invoke(url: String): Flow<Boolean> {
        return repository.isFavorite(url)
    }
}

```

**What it does:** Checks if an article is favorited (by URL) **Returns:** `Flow<Boolean>` - Emits true/false whenever status changes

---

### Repository Implementation ( `data/repository/NewsRepositoryImpl.kt` )

The repository coordinates between data sources (API and database).

```

class NewsRepositoryImpl @Inject constructor(
    private val api: NewsApiService,
    private val dao: ArticleDao
) : NewsRepository {

    // Fetch news from API
    override suspend fun getTopHeadlines(
        page: Int,
        pageSize: Int
    ): List<Article> {
        val response = api.getTopHeadlines(
            apiKey = "YOUR_API_KEY",
            page = page,
            pageSize = pageSize
        )
        // Convert DTOs to domain models
        return response.articles.map { it.toDomainArticle() }
    }

    // Get favorites from database (reactive)
    override fun getFavoriteArticles(): Flow<List<Article>> {
        return dao.getFavoriteArticles().map { entities ->
            entities.map { it.toDomainArticle() }
        }
    }

    // Save article to database
    override suspend fun saveArticle(article: Article) {
        dao.insertArticle(article.toArticleEntity())
    }

    // Delete article from database
    override suspend fun deleteArticle(article: Article) {

```

```

        dao.deleteArticle(article.toArticleEntity())
    }

    // Check if article is favorited
    override fun isFavorite(url: String): Flow<Boolean> {
        return dao.isFavorite(url).map { count -> count > 0 }
    }
}

```

## Mappers ( data/Mappers.kt )

Mappers convert between different data representations.

```

// API DTO → Domain Model
fun ArticleDto.toDomainArticle(): Article {
    return Article(
        sourceName = source.name ?: "",
        author = author ?: "Unknown",
        title = title ?: "",
        description = description ?: "",
        url = url ?: "",
        urlToImage = urlToImage ?: "",
        publishedAt = publishedAt ?: "",
        content = content ?: "",
        isFavorite = false
    )
}

// Database Entity → Domain Model
fun ArticleEntity.toDomainArticle(): Article {
    return Article(
        sourceName = sourceName ?: "",
        author = author ?: "Unknown",
        title = title ?: "",
        description = description ?: "",
        url = url,
        urlToImage = urlToImage ?: "",
        publishedAt = publishedAt ?: "",
        content = content ?: "",
        isFavorite = isFavorite
    )
}

// Domain Model → Database Entity
fun Article.toArticleEntity(): ArticleEntity {
    return ArticleEntity(
        url = url,
        sourceName = sourceName,
        author = author,
        title = title,
        description = description,
        urlToImage = urlToImage,
        publishedAt = publishedAt,
        content = content,
        isFavorite = isFavorite
    )
}

```

```
)  
}
```

## 8. Database Implementation

### Overview

The app uses **Room** - Android's recommended database library that provides an abstraction layer over SQLite.

### Database Structure

| DATABASE: news_app_db |         |                           |
|-----------------------|---------|---------------------------|
| TABLE: articles       |         |                           |
| url (PRIMARY KEY)     | TEXT    | Unique article identifier |
| sourceName            | TEXT    | News source name          |
| author                | TEXT    | Article author            |
| title                 | TEXT    | Article headline          |
| description           | TEXT    | Short description         |
| urlToImage            | TEXT    | Image URL                 |
| publishedAt           | TEXT    | Publication date          |
| content               | TEXT    | Full article content      |
| isFavorite            | INTEGER | 0 = not saved, 1 = saved  |

### Database Files Explained

#### 1. ArticleEntity ( data/db/entity/ArticleEntity.kt )

This defines the table structure:

```
@Entity(tableName = "articles")  
data class ArticleEntity(  
    @PrimaryKey                // This column is the unique identifier  
    val url: String,           // URL is unique for each article  
  
    val sourceName: String?,   // Nullable - may not always be provided  
    val author: String?,  
    val title: String?,  
    val description: String?,  
    val urlToImage: String?,  
    val publishedAt: String?,  
    val content: String?,  
    val isFavorite: Boolean = false // Default to not favorited  
)
```

#### Key Concepts:

- `@Entity` - Marks this class as a database table
- `@PrimaryKey` - Unique identifier for each row

- Nullable types ( `String?` ) - Allow null values
- 

## 2. ArticleDao ( `data/db/dao/ArticleDao.kt` )

DAO (Data Access Object) defines database operations:

```
@Dao
interface ArticleDao {

    // Insert or update an article
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertArticle(article: ArticleEntity)

    // Delete an article
    @Delete
    suspend fun deleteArticle(article: ArticleEntity)

    // Get all favorited articles (reactive - emits when data changes)
    @Query("SELECT * FROM articles WHERE isFavorite = 1")
    fun getFavoriteArticles(): Flow<List<ArticleEntity>>

    // Check if article is favorited (by URL)
    @Query("SELECT COUNT(*) FROM articles WHERE url = :url AND isFavorite = 1")
    fun isFavorite(url: String): Flow<Int>
}
```

### Key Concepts:

- `@Dao` - Marks this interface as a Data Access Object
  - `@Insert` - Insert operation
  - `@Delete` - Delete operation
  - `@Query` - Custom SQL query
  - `suspend` - Runs on background thread (coroutines)
  - `Flow` - Reactive stream that emits when data changes
  - `OnConflictStrategy.REPLACE` - If article exists, update it
- 

## 3. AppDatabase ( `data/db/AppDatabase.kt` )

Database configuration:

```
@Database(
    entities = [ArticleEntity::class], // List of all tables
    version = 1,                       // Database version
    exportSchema = false                // Don't export schema to file
)
abstract class AppDatabase : RoomDatabase() {
    abstract fun articleDao(): ArticleDao
}
```

### Key Concepts:

- `@Database` - Marks this as a Room database
  - `entities` - Array of all table classes
  - `version` - Increment when schema changes (for migrations)
  - `abstract fun articleDao()` - Provides access to the DAO
-

## How Database is Accessed

### 1. Hilt Provides Database ( di/DatabaseModule.kt ):

```
@Provides
@Singleton
fun provideDatabase(@ApplicationContext context: Context): AppDatabase {
    return Room.databaseBuilder(
        context,
        AppDatabase::class.java,
        "news_app_db"           // Database file name
    ).build()
}

@Provides
@Singleton
fun provideArticleDao(database: AppDatabase): ArticleDao {
    return database.articleDao()
}
```

### 2. Repository Uses DAO:

```
class NewsRepositoryImpl @Inject constructor(
    private val dao: ArticleDao    // Injected by Hilt
) {
    override fun getFavoriteArticles(): Flow<List<Article>> {
        return dao.getFavoriteArticles().map { entities ->
            entities.map { it.toDomainArticle() }
        }
    }
}
```

### 3. Use Case Uses Repository:

```
class GetFavoriteArticlesUseCase @Inject constructor(
    private val repository: NewsRepository
) {
    operator fun invoke() = repository.getFavoriteArticles()
}
```

### 4. ViewModel Uses Use Case:

```
class FavoritesViewModel @Inject constructor(
    private val getFavoriteArticlesUseCase: GetFavoriteArticlesUseCase
) {
    init {
        viewModelScope.launch {
            getFavoriteArticlesUseCase().collect { articles ->
                _state.value = FavoritesState(articles = articles)
            }
        }
    }
}
```

---

## Database Operations Flow

### Saving a Favorite:

```
User taps heart icon
    ↓
DetailViewModel.onFavoriteClick()
    ↓
saveArticleUseCase(article.copy(isFavorite = true))
    ↓
NewsRepositoryImpl.saveArticle(article)
    ↓
article.toArticleEntity() // Convert to entity
    ↓
dao.insertArticle(entity) // SQL: INSERT OR REPLACE INTO articles VALUES (...)
    ↓
Room notifies all observers of data change
    ↓
getFavoriteArticles() Flow emits new list
    ↓
FavoritesScreen automatically updates
```

### Checking if Favorited:

```
DetailScreen opens
    ↓
DetailViewModel.initArticle(article)
    ↓
checkFavoriteStatusUseCase(article.url)
    ↓
dao.isFavorite(url) // SQL: SELECT COUNT(*) FROM articles WHERE url = ? AND isFavorite = 1
    ↓
Flow emits count (0 or 1)
    ↓
count > 0 → isFavorite = true
    ↓
Heart icon shows filled/outline
```

---

## 9. Network Layer

### API Configuration

The app uses **NewsAPI.org** to fetch news headlines.

**Base URL:** `https://newsapi.org/v2/`

### Endpoint Used:

```
GET /top-headlines?apiKey={key}&country=us&page=1&pageSize=20
```

### Network Files Explained

#### 1. NewsApiService ( `data/api/NewsApiService.kt` )

Defines the API endpoints using Retrofit annotations:

```
interface NewsApiService {

    @GET("top-headlines")
    suspend fun getTopHeadlines(
        @Query("apiKey") apiKey: String,          // API authentication
        @Query("country") country: String = "us", // Default to US news
        @Query("page") page: Int = 1,             // Pagination
        @Query("pageSize") pageSize: Int = 20    // Results per page
    ): NewsResponse
}
```

#### Key Concepts:

- `@GET` - HTTP GET request
- `@Query` - URL query parameters
- `suspend` - Runs asynchronously with coroutines
- Returns `NewsResponse` - Retrofit deserializes JSON automatically

## 2. DTOs ( `data/model/NewsDtos.kt` )

Data Transfer Objects that match the API response structure:

```
// Main response wrapper
data class NewsResponse(
    val status: String,          // "ok" or "error"
    val totalResults: Int,       // Total available articles
    val articles: List<ArticleDto>
)

// Individual article from API
data class ArticleDto(
    val source: SourceDto,
    val author: String?,
    val title: String?,
    val description: String?,
    val url: String?,
    val urlToImage: String?,
    val publishedAt: String?,
    val content: String?
)

// News source information
data class SourceDto(
    val id: String?,
    val name: String?
)
```

## 3. NetworkModule ( `di/NetworkModule.kt` )

Provides network dependencies:

```
@Module
@InstallIn(SingletonComponent::class)
object NetworkModule {
```

```

// Logging for debugging network calls
@Provides
@Singleton
fun provideLoggingInterceptor(): HttpLoggingInterceptor {
    return HttpLoggingInterceptor().apply {
        level = HttpLoggingInterceptor.Level.BODY
    }
}

// HTTP client with logging
@Provides
@Singleton
fun provideOkHttpClient(interceptor: HttpLoggingInterceptor): OkHttpClient {
    return OkHttpClient.Builder()
        .addInterceptor(interceptor)
        .build()
}

// Retrofit instance for API calls
@Provides
@Singleton
fun provideRetrofit(client: OkHttpClient): Retrofit {
    return Retrofit.Builder()
        .baseUrl("https://newsapi.org/v2/")
        .client(client)
        .addConverterFactory(GsonConverterFactory.create())
        .build()
}

// API service implementation
@Provides
@Singleton
fun provideNewsApiService(retrofit: Retrofit): NewsApiService {
    return retrofit.create(NewsApiService::class.java)
}
}

```

## API Call Flow

```

HomeViewModel.getNews()
    ↓
getTopHeadlinesUseCase(page=1, pageSize=20)
    ↓
NewsRepositoryImpl.getTopHeadlines()
    ↓
api.getTopHeadlines(apiKey, country, page, pageSize)
    ↓
Retrofit builds URL: https://newsapi.org/v2/top-headlines?apiKey=xxx&country=us&page=1&pageSize=20
    ↓
OkHttp sends HTTP GET request
    ↓
Server responds with JSON
    ↓
Gson deserializes JSON → NewsResponse object
    ↓

```

```
Repository maps ArticleDto → Article (domain model)
    ↓
Result.success(articles) returned to ViewModel
    ↓
UI updates with articles
```

---

## 10. Dependency Injection

### What is Dependency Injection?

Instead of creating objects inside classes (tight coupling):

```
// BAD - tight coupling
class HomeViewModel {
    private val repository = NewsRepositoryImpl(
        NewsApiService(...), // Have to create these too!
        ArticleDao(...)
    )
}
```

We inject them from outside (loose coupling):

```
// GOOD - loose coupling with Hilt
@HiltViewModel
class HomeViewModel @Inject constructor(
    private val getTopHeadlinesUseCase: GetTopHeadlinesUseCase
) : ViewModel()
```

### Hilt Setup

#### 1. Application Class ( `NewsApp.kt` )

```
@HiltAndroidApp // Required for Hilt
class NewsApp : Application()
```

#### 2. MainActivity ( `MainActivity.kt` )

```
@AndroidEntryPoint // Enables injection in this Activity
class MainActivity : ComponentActivity()
```

#### 3. ViewModels

```
@HiltViewModel // Enables injection in this ViewModel
class HomeViewModel @Inject constructor(
    private val useCase: GetTopHeadlinesUseCase
) : ViewModel()
```

### DI Modules

Modules tell Hilt how to create dependencies.

#### NetworkModule

```

@Module
@InstallIn(SingletonComponent::class) // Lives for entire app lifetime
object NetworkModule {

    @Provides // Tells Hilt how to create this
    @Singleton // Only create one instance
    fun provideNewsApiService(retrofit: Retrofit): NewsApiService {
        return retrofit.create(NewsApiService::class.java)
    }
}

```

## DatabaseModule

```

@Module
@InstallIn(SingletonComponent::class)
object DatabaseModule {

    @Provides
    @Singleton
    fun provideDatabase(@ApplicationContext context: Context): AppDatabase {
        return Room.databaseBuilder(
            context,
            AppDatabase::class.java,
            "news_app_db"
        ).build()
    }
}

```

## AppModule

```

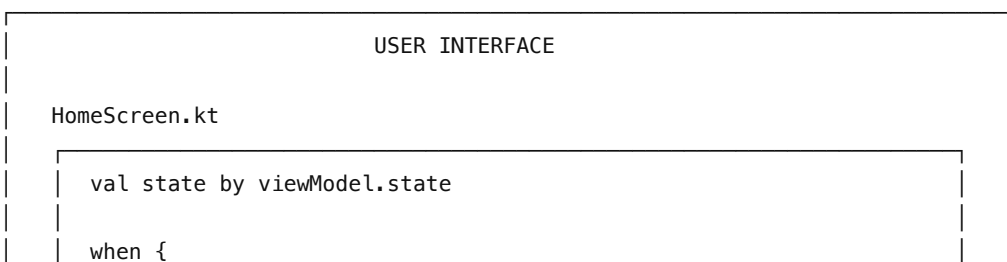
@Module
@InstallIn(SingletonComponent::class)
abstract class AppModule {

    // Binds interface to implementation
    @Binds
    @Singleton
    abstract fun bindNewsRepository(impl: NewsRepositoryImpl): NewsRepository
}

```

# 11. Data Flow Diagrams

## Complete Data Flow for Fetching News



```

state.isLoading → ShimmerLoading()
state.error != null → ErrorCard()
else → LazyColumn { items(state.articles) { ArticleItem() } }
}

```

↑ observes

## VIEWMODEL

HomeViewModel.kt

```

private val _state = mutableStateOf(HomeState())
val state: State<HomeState> = _state

init { getNews() }

private fun getNews() {
    viewModelScope.launch {
        _state.value = HomeState(isLoading = true)
        getTopHeadlinesUseCase(1, 20)
            .onSuccess { _state.value = HomeState(articles = it) }
            .onFailure { _state.value = HomeState(error = ...) }
    }
}

```

calls

## USE CASE

GetTopHeadlinesUseCase.kt

```

operator fun invoke(page: Int, pageSize: Int): Result<List<Article>>
    return try {
        val articles = repository.getTopHeadlines(page, pageSize)
        Result.success(articles)
    } catch (e: Exception) {
        Result.failure(e)
    }

```

calls

## REPOSITORY

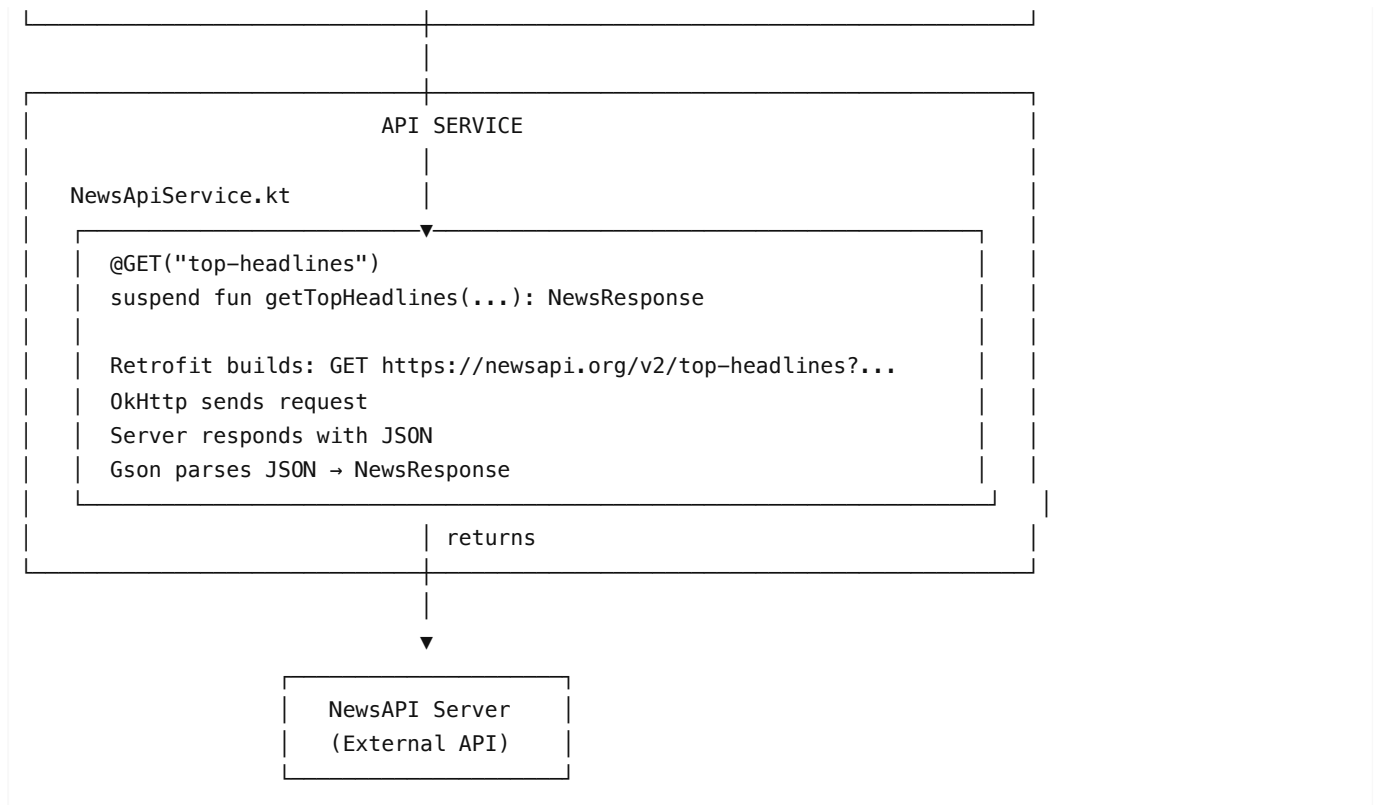
NewsRepositoryImpl.kt

```

suspend fun getTopHeadlines(page: Int, pageSize: Int): List<Article> {
    val response = api.getTopHeadlines(API_KEY, "us", page, pageSize)
    return response.articles.map { it.toDomainArticle() }
}

```

calls



## 12. How to Add New Features

### Example 1: Add a Search Feature

#### Step 1: Add API Endpoint

In `data/api/NewsApiService.kt` :

```
@GET("everything")
suspend fun searchNews(
    @Query("apiKey") apiKey: String,
    @Query("q") query: String,
    @Query("page") page: Int = 1,
    @Query("pageSize") pageSize: Int = 20
): NewsResponse
```

#### Step 2: Add Repository Method

In `domain/repository/NewsRepository.kt` :

```
suspend fun searchNews(query: String, page: Int, pageSize: Int): List<Article>
```

In `data/repository/NewsRepositoryImpl.kt` :

```
override suspend fun searchNews(query: String, page: Int, pageSize: Int): List<Article> {
    val response = api.searchNews(API_KEY, query, page, pageSize)
    return response.articles.map { it.toDomainArticle() }
}
```

#### Step 3: Create Use Case

In `domain/usecase/NewsUseCases.kt` :

```
class SearchNewsUseCase @Inject constructor(
    private val repository: NewsRepository
) {
    suspend operator fun invoke(query: String): Result<List<Article>> {
        return try {
            Result.success(repository.searchNews(query, 1, 20))
        } catch (e: Exception) {
            Result.failure(e)
        }
    }
}
```

#### Step 4: Add to ViewModel

In `viewmodel/HomeViewModel.kt` :

```
@HiltViewModel
class HomeViewModel @Inject constructor(
    private val getTopHeadlinesUseCase: GetTopHeadlinesUseCase,
    private val searchNewsUseCase: SearchNewsUseCase // Add this
) : ViewModel() {

    var searchQuery by mutableStateOf("")
    private set

    fun onSearchQueryChange(query: String) {
        searchQuery = query
    }

    fun searchNews() {
        viewModelScope.launch {
            _state.value = _state.value.copy(isLoading = true)
            searchNewsUseCase(searchQuery)
                .onSuccess { articles ->
                    _state.value = _state.value.copy(
                        isLoading = false,
                        articles = articles
                    )
                }
                .onFailure { error ->
                    _state.value = _state.value.copy(
                        isLoading = false,
                        error = error.message
                    )
                }
        }
    }
}
```

#### Step 5: Add UI

In `ui/home/HomeScreen.kt` :

```

@Composable
fun HomeScreen(...) {
    Column {
        // Add search bar
        OutlinedTextField(
            value = viewModel.searchQuery,
            onChange = { viewModel.onSearchQueryChange(it) },
            label = { Text("Search news...") },
            trailingIcon = {
                IconButton(onClick = { viewModel.searchNews() }) {
                    Icon(Icons.Default.Search, "Search")
                }
            }
        )

        // Rest of the screen...
    }
}

```

## Example 2: Add Categories/Topics

### Step 1: Define Categories

Create `domain/model/NewsCategory.kt` :

```

enum class NewsCategory(val apiValue: String) {
    GENERAL("general"),
    BUSINESS("business"),
    TECHNOLOGY("technology"),
    SPORTS("sports"),
    ENTERTAINMENT("entertainment"),
    HEALTH("health"),
    SCIENCE("science")
}

```

### Step 2: Update API Service

In `data/api/NewsApiService.kt` :

```

@GET("top-headlines")
suspend fun getTopHeadlines(
    @Query("apiKey") apiKey: String,
    @Query("country") country: String = "us",
    @Query("category") category: String? = null, // Add this
    @Query("page") page: Int = 1,
    @Query("pageSize") pageSize: Int = 20
): NewsResponse

```

### Step 3: Update Repository

```

suspend fun getTopHeadlines(
    page: Int,
    pageSize: Int,

```

```

        category: NewsCategory? = null
    ): List<Article>

```

#### Step 4: Add Category Chips UI

```

@Composable
fun CategoryChips(
    selectedCategory: NewsCategory?,
    onCategorySelected: (NewsCategory?) -> Unit
) {
    LazyRow {
        items(NewsCategory.values()) { category ->
            FilterChip(
                selected = category == selectedCategory,
                onClick = { onCategorySelected(category) },
                label = { Text(category.name) }
            )
        }
    }
}

```

### Example 3: Add Article Sharing

#### Step 1: Add Share Function

In `ui/detail/DetailScreen.kt` :

```

@Composable
fun DetailScreen(...) {
    val context = LocalContext.current

    fun shareArticle(article: Article) {
        val intent = Intent(Intent.ACTION_SEND).apply {
            type = "text/plain"
            putExtra(Intent.EXTRA_SUBJECT, article.title)
            putExtra(Intent.EXTRA_TEXT, "${article.title}\n\n${article.url}")
        }
        context.startActivity(Intent.createChooser(intent, "Share article"))
    }

    // Add share icon to top bar
    IconButton(onClick = { article?.let { shareArticle(it) } }) {
        Icon(Icons.Default.Share, "Share")
    }
}

```

## 13. Common Patterns Used

### 1. State Hoisting

State is "hoisted" up to the ViewModel, making composables stateless and easier to test:

```

// ViewModel holds state
class HomeViewModel {

```

```

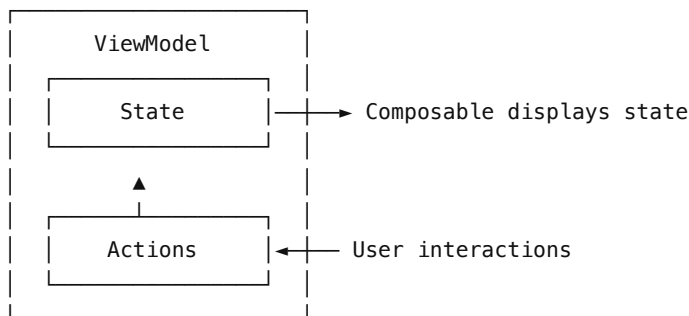
    var state by mutableStateOf(HomeState())
}

// Composable receives state and actions
@Composable
fun HomeScreen(
    state: HomeState,          // State passed down
    onArticleClick: (Article) -> Unit // Action passed up
)

```

## 2. Unidirectional Data Flow

Data flows down, events flow up:



## 3. Repository Pattern

Single source of truth for data:

```

interface NewsRepository {
    // Define what operations are available
    suspend fun getTopHeadlines(): List<Article>
    fun getFavoriteArticles(): Flow<List<Article>>
}

class NewsRepositoryImpl : NewsRepository {
    // Implement using actual data sources
    override suspend fun getTopHeadlines() = api.getHeadlines()
}

```

## 4. Use Case Pattern

Each use case = one business operation:

```

class GetTopHeadlinesUseCase {
    operator fun invoke() = repository.getTopHeadlines()
}

// Usage in ViewModel
getTopHeadlinesUseCase() // Reads like plain English!

```

## 5. Optimistic Updates

Update UI immediately, sync database in background:

```
fun onFavoriteClick() {
    // 1. Update UI immediately (optimistic)
    _state.value = _state.value.copy(isFavorite = true)

    // 2. Update database in background
    viewModelScope.launch {
        saveArticleUseCase(article)
    }
}
```

## 14. Troubleshooting

### Common Issues

#### 1. API Key Invalid

**Error:** 401 Unauthorized **Solution:** Get a new API key from [newsapi.org](https://newsapi.org) and update `NetworkModule.kt`

#### 2. Network Error

**Error:** `UnknownHostException` **Solution:**

- Check internet connection
- Verify `INTERNET` permission in `AndroidManifest.xml`

#### 3. Database Migration Error

**Error:** `Room cannot verify data integrity` **Solution:**

- Uninstall app and reinstall (loses data)
- Or add proper migration in `AppDatabase.kt`

#### 4. Hilt Injection Error

**Error:** `Cannot create instance of ViewModel` **Solution:**

- Ensure `@HiltViewModel` is on `ViewModel`
- Ensure `@AndroidEntryPoint` is on `Activity`
- Ensure `@Inject` is on constructor

#### 5. Compose Not Updating

**Symptom:** UI doesn't reflect state changes **Solution:**

- Ensure using `mutableStateOf()` or `StateFlow`
- Use `by` delegate for automatic recomposition
- Check that state is actually being updated

## Appendix A: Key Files Quick Reference

| File            | Purpose                        |
|-----------------|--------------------------------|
| MainActivity.kt | App entry point                |
| NewsApp.kt      | Application class (Hilt setup) |
| NavGraph.kt     | Navigation setup               |
| Screen.kt       | Route definitions              |

|                       |                            |
|-----------------------|----------------------------|
| HomeViewModel.kt      | Home screen logic          |
| DetailViewModel.kt    | Detail screen logic        |
| NewsRepository.kt     | Data access interface      |
| NewsRepositoryImpl.kt | Data access implementation |
| ArticleDao.kt         | Database operations        |
| NewsApiService.kt     | API endpoints              |
| Mappers.kt            | Data conversion            |

## Appendix B: Gradle Dependencies

```
// Core Android
androidx.core:core-ktx
androidx.appcompat:appcompat
androidx.lifecycle:lifecycle-runtime-ktx

// Compose
androidx.compose.bom (Bill of Materials)
androidx.compose.ui:ui
androidx.compose.material3:material3
androidx.navigation:navigation-compose

// Hilt (Dependency Injection)
com.google.dagger:hilt-android
androidx.hilt:hilt-navigation-compose

// Room (Database)
androidx.room:room-runtime
androidx.room:room-ktx

// Retrofit (Networking)
com.squareup.retrofit2:retrofit
com.squareup.retrofit2:converter-gson

// Coil (Image Loading)
io.coil-kt:coil-compose
```

## Conclusion

This documentation covers the complete News application from start to finish. As a beginner, you should now understand:

1. **How the app is structured** - Clean Architecture with clear separation of concerns
2. **How data flows** - From API/Database through Repository, Use Cases, ViewModels to UI
3. **How navigation works** - Using Jetpack Navigation Compose
4. **How the database works** - Room with Entity, DAO, and Database
5. **How networking works** - Retrofit with DTOs and mappers
6. **How dependency injection works** - Hilt modules and annotations
7. **How to add new features** - Following the established patterns

Use this documentation as your reference while exploring the codebase and building new features!

